Final Project

Data Science in Cyber

Website Phishing Detection:

Reproduction, Evaluation, and Critical Analysis of a Kaggle Machine Learning Project

Submitted by: Shada Mazzawi

ID: 212931554

1. Summary of the Source

The Problem Being Addressed

This project is about detecting phishing websites automatically. The goal is to classify websites as phishing, suspicious, or legitimate based on pre-extracted features. Phishing websites copy the look of trusted services to steal sensitive information like passwords and credit card details.

Why the Problem is Important

Phishing is a major cybersecurity threat because it is often the first step in credential theft, financial fraud, malware installation, and broader organizational compromise. Manual review cannot keep up with the volume of new phishing websites created every day. Machine learning is useful in this setting because it can learn patterns from previously labeled websites and apply those patterns to new cases quickly.

The Proposed Solution

The source evaluated in this project is the Kaggle notebook "Website Phishing Detection ML Project" by duygujones. The notebook loads the file Website Phishing.csv from the Kaggle dataset website-phishing-data-set and applies a supervised machine learning workflow. The author inspects the data, normalizes the features using MinMax scaling, trains classification models, and compares their performance.

The Dataset Used

The dataset contains 1,353 website records, 9 predictive features, and one target column named Result. The target has three encoded classes: -1 for phishing, 0 for suspicious, and 1 for legitimate. The nine predictive features are SFH, popUpWidnow, SSLfinal_State, Request_URL, URL_of_Anchor, web_traffic, URL_Length, age_of_domain, and having_IP_Address. All features are encoded using small integer values such as -1, 0, and 1, so they should be interpreted as categorical or ordinal indicators rather than raw continuous measurements.

Property	Value
Rows	1,353
Predictive features	9
Target classes	3
Phishing class	-1
Suspicious class	0
Legitimate class	1

The Model and Methodology Employed

The original approach uses supervised classification. This reproduction kept the same dataset and extended the analysis with more thorough data inspection, correlation analysis, duplicate checks, cross-validation, additional metrics, feature importance, and error analysis. Logistic Regression, Decision Tree, and Random Forest models were trained. Random Forest was used as the main non-linear ensemble model.

2. Critical Evaluation

The Main Claims Made by the Author

Looking at the notebook, the author makes three main claims: the features are good enough to separate the three website classes; machine learning models can achieve high performance on this dataset; and normalizing features before training is the right preprocessing choice.

Whether the Claims are Supported by the Evidence

The general claim is supported by the results of this reproduction. Random Forest achieved 88.93% accuracy, 88.84% weighted F1, 0.8025 MCC, and 97.52% weighted one-vs-rest ROC-AUC on the 20% test split. These results show that the features contain useful predictive signal. However, the results are not perfect and should not be interpreted as evidence of a deployment-ready phishing detection system.

Whether the Evaluation Methodology is Appropriate

The original evaluation has a few gaps. A single train/test split is not very stable on a dataset this small, especially with only 103 suspicious-class samples. 5-fold stratified cross-validation, MCC, F2 score, multi-class ROC-AUC, and class-specific error analysis were therefore added. These provide a more reliable and cybersecurity-relevant evaluation.

Possible Weaknesses and Limitations

- The dataset is quite small — only 1,353 records and 9 features.
- The suspicious class is much smaller than the phishing and legitimate classes, which makes it harder to learn.
- There are 629 exact duplicate encoded rows. Duplicate rows can make random train/test performance optimistic if similar patterns appear in both sets.
- There are duplicate feature patterns with conflicting labels, meaning the same feature vector can correspond to different classes. This limits the maximum achievable accuracy.
- The dataset does not include raw URLs or collection timestamps, so temporal generalization and realistic deployment behavior cannot be fully tested.

Whether the Conclusions are Justified

The conclusion that machine learning can help detect phishing websites is justified for this dataset. Stronger claims about real-world deployment would require larger data, temporal validation, clearer treatment of duplicates, and evaluation under realistic class prevalence.

One finding from this reproduction directly contradicts the implied claim of strong model performance. The original notebook does not perform any duplicate analysis, and the reported results are based on a raw dataset containing 629 exact duplicate rows. When these duplicate rows are removed and the experiment is repeated on the deduplicated dataset, Random Forest accuracy drops from 88.93% to 79.31%, weighted F1 drops from 88.84% to 78.87%, and MCC drops from 0.8025 to 0.6336. This is a reduction of nearly 10 percentage points in accuracy and nearly 0.17 in MCC. This means the strong performance reported in the original notebook is partly an artifact of duplicate rows appearing in both the training and test sets after a random split, not purely a result of the model learning meaningful phishing patterns. A second partial contradiction concerns the feature sufficiency claim. The analysis revealed that some identical feature patterns appear with different class labels in the dataset. This means the nine features do not fully determine the correct class — a fundamental limitation that the original author never acknowledges. These two findings do not invalidate the general approach, but they do mean the original performance claims should be treated with caution rather than accepted at face value.

Exploratory Data Analysis

Feature Distributions

All nine features take encoded integer values of -1, 0, or 1, representing negative, neutral, and positive phishing indicators respectively. Histograms of each feature show that most features are skewed toward

specific values, reflecting the structured nature of the encoding. For example, having_IP_Address is 0 for most legitimate sites, while phishing sites show a higher proportion of 1 (IP address used).

Missing Values and Class Imbalance

No missing values were found in the dataset. The class distribution consists of 702 legitimate websites (1), 548 phishing websites (-1), and 103 suspicious websites (0). The suspicious class is the smallest class in the dataset. This class imbalance is an important finding because classifiers tend to perform worse on minority classes. In the real world, suspicious websites often represent newly registered or ambiguous sites that are difficult to classify reliably with relatively few training examples. Therefore, weighted evaluation metrics (F1, F2, and MCC) were used throughout the analysis to provide a fairer assessment of model performance.

Outlier Analysis

Because all features are integer-encoded with values restricted to {-1, 0, 1}, traditional outlier detection methods such as the IQR or z-score are not applicable because there are no continuous distributions to analyze. Instead, category frequencies were examined. Features such as popUpWidnow showed a very high concentration in one category (over 90% of the samples), making them low-variance features.

Temporal Analysis

The dataset does not contain explicit timestamp or collection-date columns. The feature age_of_domain serves as a temporal proxy — it reflects how old the domain registration is, which is relevant because phishing sites use freshly registered domains. No time-series analysis is applicable. This project is treated as a static classification task.

Group-By and Crosstab Analysis

A group-by analysis comparing feature means across the three classes shows that phishing sites consistently have lower SFH scores, lower SSLfinal_State scores, lower URL_of_Anchor scores, and higher having_IP_Address values than legitimate sites. The suspicious class falls between legitimate and phishing on most features, which explains why it is the hardest class to classify correctly.

Correlation Analysis — Pearson, Spearman, and Kendall

Three correlation methods were computed and compared. Pearson correlation measures the linear relationship between two variables. Formula: r = cov(X,Y) / (sigma_X * sigma_Y). It assumes both variables are continuous and normally distributed. Advantage: easy to interpret. Limitation: not appropriate for ordinal or non-normal data. Spearman correlation measures the monotonic relationship using rank ordering of values. It makes no distributional assumptions and is robust to outliers and skewed data. Advantage: suitable for ordinal data. Limitation: loses information about the magnitude of differences. Kendall correlation measures rank concordance — the proportion of concordant minus discordant pairs. It is preferred for small datasets and when rank consistency is the primary interest. Advantage: robust for small samples. Limitation: computationally heavier. For this dataset, all nine features are integer-encoded as -1, 0, or 1, making them ordinal rather than continuous. Pearson is therefore technically inappropriate but was computed for comparison. Spearman was selected as the primary method because it correctly handles ordinal data and is robust to the skewed suspicious-class distribution. All three methods agreed on the direction of most correlations, which increases confidence in the findings. The strongest correlates with the phishing class were SSLfinal_State, URL_of_Anchor, Request_URL, and having_IP_Address.

3. Feature Engineering Analysis

Whether Feature Engineering Was Performed

The dataset already comes with pre-extracted features — no raw URLs or page content to process. The original notebook mainly just applies MinMax scaling on top of that. In this reproduction, the feature encoding was analyzed, redundancy and duplicates were checked, interpretable features were preserved, and scaling was used where appropriate.

Which Features Were Used

All nine original features were used: SFH, popUpWidnow, SSLfinal_State, Request_URL, URL_of_Anchor, web_traffic, URL_Length, age_of_domain, and having_IP_Address. These features represent form handling, popup behavior, SSL state, URL/request behavior, anchor URLs, traffic/reputation, URL length, domain age, and whether an IP address is used.

Transformations Applied

MinMax scaling was applied inside the Logistic Regression pipeline. This transformation maps each feature to the range [0, 1] using x_scaled = (x - x_min) / (x_max - x_min). Scaling is important for Logistic Regression because it is optimization-based and can be affected by feature scale. Tree-based models such as Decision Tree and Random Forest do not require scaling, so they were trained directly on the encoded features.

The target variable was not converted to a binary label because the original dataset is a three-class classification problem. Keeping the three-class target preserves the original problem structure and allows explicit analysis of the suspicious class.

Redundancy in the System

No constant or identical feature columns were found. However, redundancy can appear in a subtler form: multicollinearity, where two or more features carry overlapping information. In this dataset, URL_Length and having_IP_Address may be correlated, since phishing URLs that embed a raw IP address tend to be longer. Similarly, Request_URL and URL_of_Anchor both measure how much page content points outside the legitimate domain, so they may overlap. To spot multicollinearity formally, a Variance Inflation Factor (VIF) analysis would be appropriate. VIF measures how much the variance of a feature is inflated due to its linear relationship with other features. A VIF above 5 is considered moderate and above 10 is problematic. Alternatively, a pairwise correlation matrix with a threshold of 0.85 can identify redundant feature pairs. To tackle multicollinearity, the options are: drop the less informative feature, apply PCA to collapse correlated features into uncorrelated components, or use a tree-based model such as Random Forest, which selects features based on information gain and is naturally robust to multicollinearity. In this project, Random Forest was used as the primary model partly for this reason. The duplicate rows in this dataset are a deeper problem than ordinary feature redundancy — identical feature patterns with different class labels mean the model cannot always learn a consistent decision boundary, regardless of which features are selected.

Whether the Feature Engineering Was Meaningful

The features are meaningful from both a cybersecurity and a mathematical perspective. SSLfinal_State captures whether a site uses a valid SSL certificate from a trusted authority. Phishing sites often use no SSL or a self-signed certificate, making this a strong phishing indicator. Mathematically it is an ordinal feature encoded as -1, 0, or 1, capturing three levels of trust. SFH (Server Form Handler) describes where a webpage's login form sends collected data. If the form submits to a different domain, this is a classic phishing technique for credential harvesting. Request_URL measures what fraction of the page's embedded objects (images, scripts) are loaded from external domains. Phishing pages often pull resources from many different servers, giving a high ratio. URL_of_Anchor measures what fraction of anchor links point away from the legitimate

domain. Phishing sites frequently embed links to legitimate websites to appear trustworthy while the malicious form is hosted elsewhere. The feature `age_of_domain` indicates whether a domain is relatively new or old using encoded values. Phishing campaigns often rely on recently registered domains to avoid reputation-based detection, making a young domain age an important phishing indicator. The feature `having_IP_Address` identifies URLs that use a raw IP address instead of a domain name. This is a common evasion technique because IP-based URLs bypass many domain-reputation mechanisms.

From a mathematical perspective, all features are represented using low-cardinality integer encodings (-1, 0, 1 or 0, 1). These variables should therefore be interpreted as ordinal rather than continuous, which motivates the use of rank-based correlation measures and explains why tree-based models perform better than simple linear models. Although the compact encoding improves interpretability, the limited number of possible values also restricts the amount of information each individual feature can provide.

Additional Features That Could Improve Performance

Features that could improve performance include raw URL text, SSL certificate age and issuer, redirect chain length, domain registration metadata, typosquatting distance from known brands, and threat intelligence scores from services like VirusTotal.

4. Reproducibility Analysis

Whether the Code Can Be Executed Successfully

The original notebook runs fine on Kaggle. Moving it to Google Colab required downloading the CSV file manually and uploading it, which the author does not document anywhere. The rebuilt notebook is designed to accept either Website Phishing.csv or a ZIP file containing it.

Whether All Required Files and Dependencies Are Available

The required dataset is publicly available through Kaggle. The Python dependencies are standard packages: pandas, numpy, matplotlib, and scikit-learn. No special hardware is required. A requirements file would improve reproducibility because scikit-learn behavior can vary slightly across versions.

Whether Hidden Preprocessing Steps Exist

One thing worth noting is that the dataset is already encoded when loaded — the actual feature extraction process that produced these values is not shown or documented anywhere in the notebook. The original notebook treats the encoded values as ready-to-use numerical features. This analysis makes these issues explicit and checks whether the encoding creates quality issues such as duplicates or conflicting labels.

Overall Reproducibility Assessment

The work is moderately reproducible. The code and dataset are accessible, but the small dataset, duplicate records, encoded feature definitions, and lack of full feature-extraction code limit complete reproducibility. The general approach can be reproduced, but the exact interpretation of feature meanings depends on the dataset documentation.

5. Experimental Results

The Experiments Performed

The experiment included data loading and inspection, missing-value analysis, duplicate analysis, class imbalance analysis, feature distribution analysis, temporal proxy analysis using age_of_domain, outlier and encoded-value validation, correlation comparison using Pearson, Spearman, and Kendall, feature engineering, model training, cross-validation, feature importance, and error analysis.

Models Trained

Three models were trained: Logistic Regression, Decision Tree, and Random Forest. Logistic Regression provides a simple interpretable baseline. Decision Tree provides a simple non-linear model. Random Forest provides a stronger ensemble model and was selected as the main recommended model.

Evaluation Metrics

The evaluation uses accuracy, weighted precision, weighted recall, weighted F1, weighted F2, MCC, weighted one-vs-rest ROC-AUC, confusion matrices, and class-specific classification reports. Weighted averages are used because the suspicious class is much smaller than the phishing and legitimate classes. MCC is included because it summarizes all cells of the confusion matrix and is more informative than accuracy alone.

- Accuracy = (TP + TN) / (TP + TN + FP + FN). It measures the overall proportion of correct predictions, but it can be misleading when classes are imbalanced.
- Precision = TP / (TP + FP). In phishing detection, low precision means the model falsely flags many non-phishing websites as phishing, which can create user frustration and unnecessary manual review.
- Recall = TP / (TP + FN). For the phishing class, recall is especially important because a false negative means a phishing website is missed and a user may be exposed to credential theft or malware.
- F1 Score = 2 * (precision * recall) / (precision + recall). It balances precision and recall equally.
- F2 Score = (1 + 2^2) * (precision * recall) / (2^2 * precision + recall). It gives more weight to recall than precision, which is appropriate in cybersecurity because missing a phishing website is usually more dangerous than raising a false alarm.
- Matthews Correlation Coefficient (MCC) is a balanced correlation-like measure between the true and predicted labels. It considers all parts of the confusion matrix and is more informative than accuracy when class sizes differ.
- ROC-AUC measures how well the model separates classes across decision thresholds. Because this is a three-class problem, one-vs-rest ROC-AUC with weighted averaging is reported.

Main Test Results

Metric	Logistic Regression	Decision Tree	Random Forest
Accuracy	83.76%	88.93%	88.93%
Weighted Precision	77.92%	89.11%	88.87%
Weighted Recall	83.76%	88.93%	88.93%
Weighted F1	80.70%	88.96%	88.84%
Weighted F2	82.50%	88.93%	88.88%
MCC	0.7040	0.8054	0.8025
Weighted ROC-AUC	94.32%	92.48%	97.52%

Random Forest and Decision Tree achieved the highest accuracy, while Random Forest achieved the strongest ROC-AUC. Logistic Regression performed worse, suggesting that the relationship between the encoded features and website class is not purely linear.

Random Forest Confusion Matrix

Actual \ Predicted	Phishing	Suspicious	Legitimate
Phishing	127	2	11
Suspicious	3	14	4
Legitimate	8	2	100

The Random Forest model missed 13 phishing-related cases in the test split: 2 phishing websites were predicted as suspicious and 11 were predicted as legitimate. In cybersecurity, phishing false negatives are the most dangerous errors because they allow malicious websites to pass undetected.

Cross-Validation Results

Model	5-fold CV Accuracy Mean	Std. Dev.
Logistic Regression	82.85%	1.57%
Decision Tree	87.95%	2.02%
Random Forest	89.28%	1.74%

Duplicate Sensitivity Analysis

After exact duplicate rows were removed, the dataset decreased from 1,353 rows to 724 rows. On this deduplicated version, Random Forest accuracy decreased to 79.31%, weighted F1 to 78.87%, MCC to 0.6336, and ROC-AUC to 91.31%. Duplicate rows can inflate estimated performance because identical feature patterns may appear in both the training and test sets after a random split. This confirms that the raw-data performance is likely optimistic.

Feature Importance

Feature	Random Forest Importance
sfh	0.200
sslfinal_state	0.186
request_url	0.148
url_of_anchor	0.147
url_length	0.120
popupwidnow	0.078
web_traffic	0.075
age_of_domain	0.033
having_ip_address	0.013

The most important features are SFH, SSLfinal_State, Request_URL, and URL_of_Anchor. These features are meaningful because phishing sites often manipulate forms, SSL behavior, requests, and links to deceive users and collect credentials.

Error Analysis

The two types of errors have very different consequences in a phishing detection system. A False Positive occurs when a legitimate website is flagged as phishing or suspicious. The practical consequence is that a real user trying to access a legitimate service such as a bank portal, a utility website, or a government service is blocked or warned away. This causes user frustration, loss of productivity, and generates IT support tickets. The harm is annoying but recoverable — the user can try again or contact support. A False Negative occurs when a genuine phishing website slips through the filter and is classified as legitimate or suspicious. The consequences here are catastrophic: a user lands on the phishing page, submits their credentials, and the attacker harvests usernames, passwords, credit card numbers, or banking details. This can lead to financial loss, identity theft, and in corporate environments, a full network breach. The harm is often irreversible. This

asymmetry means that in phishing detection, False Negatives are far more dangerous than False Positives. A system that misses phishing sites is much more harmful than one that occasionally over-blocks legitimate sites. This justifies prioritizing Recall over Precision, and using F2 score (which weights Recall twice as heavily as Precision) rather than standard F1 or raw Accuracy. MCC is also reported because it provides a balanced view that penalizes both error types, which is appropriate when the cost of errors is asymmetric. In this reproduction, the most common errors involved the Suspicious class, which was both underrepresented and ambiguous in its feature patterns. For real deployment, a threshold tuned to maximize phishing Recall at an acceptable False Positive rate would be the recommended approach.

6. Conclusions

Key Findings

The reproduction confirmed that machine learning can classify phishing websites reasonably well on this dataset. Random Forest was the best-performing model, achieving the highest accuracy, MCC, and ROC-AUC among the three models tested. However, the dataset has real limitations: it is small, encoded, contains many duplicate rows, and some feature patterns have conflicting labels across different target classes.

Lessons Learned

The main thing this project taught is that a good accuracy number does not tell the whole story. Checking data quality, duplicates, class balance, and error patterns matters a lot in cybersecurity. A model can perform well numerically while still having weaknesses that matter in deployment, especially false negatives for phishing websites.

Strengths and Weaknesses

The clearest strengths are the interpretable features and the fact that tree-based models work well here. The main weaknesses are the small size, the tiny suspicious class, the duplicate rows, and the absence of raw URLs or timestamps.

Suggestions for Future Improvements

Going forward, using a larger and more recent dataset would help a lot. Removing or controlling for duplicate rows, collecting raw URLs, tuning the classification threshold for better phishing recall, and testing on realistic class prevalence would all make the evaluation more meaningful.

7. Executive Summary

This project reproduced and critically evaluated the Kaggle notebook "Website Phishing Detection ML Project" by duygujones. The dataset used is the Website Phishing dataset: 1,353 records, 9 encoded features, and a 3-class target (phishing, suspicious, legitimate).

The full machine learning workflow was reproduced and extended with data quality inspection, duplicate analysis, class imbalance analysis, Pearson/Spearman/Kendall correlation comparison, 5-fold cross-validation, feature importance analysis, and error analysis. Three models were trained: Logistic Regression, Decision Tree, and Random Forest.

Random Forest achieved 88.93% accuracy, 0.8025 MCC, and 97.52% weighted ROC-AUC. However, after removing 629 exact duplicate rows, accuracy dropped to 79.31% and MCC to 0.6336. This shows the raw performance is optimistic. The most important features are SFH, SSLfinal_State, Request_URL, and URL_of_Anchor — directly related to how phishing sites manipulate forms, SSL, and links.

The original notebook omits duplicate analysis, class imbalance discussion, feature engineering justification, and detailed error analysis. These omissions reduce the transparency and reproducibility of the original work.

8. Summing It Up

The Problem Being Addressed

Automated phishing website detection using supervised machine learning. Phishing websites steal sensitive information by impersonating trusted services, and manual detection cannot keep up with the volume of new phishing sites.

The Selected Article

"Website Phishing Detection ML Project" by duygujones, Kaggle, 2025. The notebook applies supervised classification to the Website Phishing dataset.

The Dataset Used

Website Phishing Dataset (Ahmed Nour). It contains 1,353 records, 9 encoded features, and a 3-class target. The dataset contains 629 exact duplicate rows and 48 feature patterns with conflicting labels.

The Methodology Employed

Data loading and inspection, missing value analysis, duplicate and conflicting-label analysis, class balance analysis, EDA with crosstabs and correlation (Pearson/Spearman/Kendall), MinMax scaling for Logistic Regression, stratified 80/20 split with random seed 42, training of three models, evaluation with accuracy/precision/recall/F1/F2/MCC/ROC-AUC, 5-fold cross-validation, duplicate sensitivity analysis, feature importance, and error analysis.

The Main Findings of the Reproduction Study

The most important finding is that 629 duplicate rows inflate the reported performance. After removing duplicates, Random Forest accuracy drops from 88.93% to 79.31% and MCC from 0.8025 to 0.6336 — a reduction of nearly 10 percentage points. 48 feature patterns also appear with conflicting labels, meaning the features cannot always determine the correct class. The original notebook reports none of these findings.

Whether the Author's Claims Were Supported

The general claim that machine learning can detect phishing websites is supported. However, the implied claim of strong model performance is partially contradicted by the duplicate sensitivity analysis. True performance on deduplicated data is substantially lower.

The Most Important Insights Obtained

Data quality inspection matters more than model selection. Duplicate rows, conflicting labels, and a small minority class all affect what evaluation numbers actually mean. In cybersecurity, false negatives are far more dangerous than false positives, so Recall and F2 are more meaningful than raw Accuracy.

Recommendation

The general approach is recommended as an educational starting point. It should not be deployed without duplicate removal, larger and more recent data, threshold optimization for phishing recall, and evaluation under realistic class prevalence.

The Final Conclusion

The reproduction confirms the usefulness of machine learning for phishing detection but reveals that the original notebook's performance claims are partly optimistic due to duplicate records. A rigorous phishing detection system requires careful data quality checks, duplicate-aware evaluation, and cybersecurity-oriented metric selection beyond raw accuracy.

References

[1] duygujones. Website Phishing Detection ML Project. Kaggle Notebook, 2025. Available at: kaggle.com/code/duygujones/website-phishing-detection-ml-project

[2] Ahmed Nour. Website Phishing Data Set. Kaggle Dataset. Available at: kaggle.com/datasets/ahmednour/website-phishing-data-set